Rúbrica proyecto de prácticas FPII 2025/26

La práctica se valora sobre 10 puntos. La corrección tendrá en cuenta el funcionamiento del programa, la coherencia entre UML e implementación y la defensa realizada.

1. Techos de nota

- No compila: máximo 3.00.
- Uso de estructuras explícitamente prohibidas: máximo 3.00.
- No se supera la defensa: máximo 3.00.

2. Penalizaciones globales

- El diagrama UML no se corresponde con el código implementado.
- Limpieza general (código comentado, muerto o de pruebas).
- Nombres o formato muy deficientes.
- Más de un `return` en una función.
- Uso de `break` fuera de `switch`.
- Uso de `continue`.

3. Diagrama UML - 3 puntos

3.1. Relaciones - 0.5

- Relaciones entre las clases principales.
- Tipo de relación correctamente representado.
- Multiplicidad cuando corresponda.

3.2. Clases - 0.5

- Clases correctamente identificadas.
- Ausencia de clases innecesarias o irrelevantes.
- Tipo correcto de elemento en UML (clase, interfaz, etc.).

3.3. Contenido - 2

- Atributos y métodos principales de cada clase
- Modificadores de visibilidad.
- Tipos, parámetros y retornos.
- Excepciones lanzadas por los métodos.

4. Código - 7 puntos

4.1. Main - 1 punto

- Creación y utilización correcta de objetos.
- Lectura/carga inicial de ficheros integrada correctamente en el arranque, aunque esté delegada en funciones auxiliares o en otra clase.
- Separación del `main` en funciones auxiliares cuando corresponde, evitando un main monolítico.
- Ausencia de lógica de dominio en `main`.

4.2. Excepciones - 1 punto

- Creación de excepciones propias necesarias.
- Lanzamiento de excepciones en el lugar adecuado.
- Captura de excepciones en el lugar adecuado.
- Capturas que no oculten errores ni vacíen el flujo del programa (`catch` genérico, `catch` vacío, sin feedback, etc.).
- Gestión de excepciones de entrada/salida.

4.3. Implementación general - 5 puntos

4.3.1. Navegación por menús - 0.70

- Clase `Menu` bien modularizada en métodos auxiliares.
- Delegación correcta en las clases del sistema, respetando la encapsulación.
- Flujo de uso comprensible para el usuario.
- Lógica de navegación sencilla y eficiente, sin duplicar código innecesariamente.

4.3.2. Equipar y desequipar - 1.20

- Movimiento correcto entre inventario y equipo.
- Reemplazo de equipo sin perder ni duplicar objetos.
- Actualización coherente de ataque y defensa.
- Uso de `IEquipable` y polimorfismo.
- Distinción correcta entre arma y armaduras.
- Encapsulación entre personaje, inventario y equipo.

4.3.3. Comprar en tienda - 1.00

- Movimiento correcto del objeto desde la tienda al inventario del personaje.
- Actualización coherente del dinero del personaje y del precio del objeto.
- La tienda deja de ofrecer el objeto comprado.
- Conservación de la información completa del objeto comprado, incluido metal.
- Encapsulación entre menú, tienda, personaje e inventario.

4.3.4. Guardar personaje - 0.70

- Escritura del estado actual del personaje en fichero.
- Conservación de datos principales, equipo e inventario.
- Formato coherente con el formato de lectura.
- Uso adecuado de escritura de ficheros sin mezclar responsabilidades.
- Encapsulación: el guardado no debe requerir manipular desde fuera las estructuras internas del personaje.

4.3.5. Uso de `toString` - 0.20

- Sobrescritura correcta de `toString` en las clases donde aporta información.
- Salida legible y útil para revisar el estado del sistema.
- Utilización de `toString` de forma adecuada en las clases.

4.3.6. Herencia de `Objeto` - 1.10

- `Objeto` como clase base.
- Herencia correcta en las clases de objetos (armas, armaduras, etc.).
- Atributos y comportamiento común ubicados en la clase base.
- Constructores bien encadenados.
- Ausencia de duplicación innecesaria entre clases hijas.

4.3.7. Uso de `enum` - 0.10

- Uso cuando hay conjunto cerrado de valores.
- Integración real en la lógica.

5. Defensa

La defensa forma parte de la evaluación. El alumnado debe poder explicar las decisiones de diseño e implementación tomadas en su práctica de forma clara y razonada. Se valorará la capacidad de argumentación y la comprensión profunda del código y el diseño, así como la capacidad de responder a preguntas sobre posibles mejoras o alternativas.